

# 自己紹介

名前：永松 涼華(ナガマツ スズカ)

趣味：プラモ/読書/ゲーム

出身：福岡情報ITクリエイター専門学校

メール:suzukareika121616@gmail.com

スキル



C/C++  
C#

2年  
1年



GitHub 2年



Photoshop

3年



Word/Excel/PowerPoint 6年



## もくじ

3~7 Ascensio Quest

8 MonsterCrusher

9 魔族の森

# 最終制作作品

• GitHub

<https://github.com/Remy-1216/MonsterCrusherMod.2>

• youtube

<https://youtu.be/LFiuKGuRJk4>

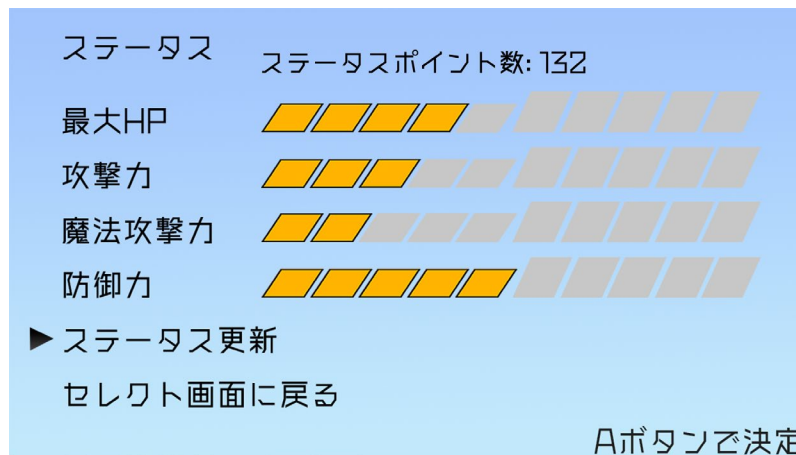
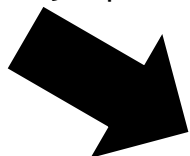
|      |                |
|------|----------------|
| 作品名  | Ascensio Quest |
| ジャンル | 3Dアクションゲーム     |
| 開発環境 | C++,DxLib      |
| 対応機種 | Windows        |
| 制作期間 | 2024/10/1～     |
| 担当   | モデル・モーション以外すべて |



# ゲーム内容



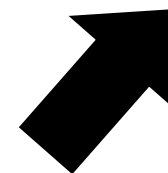
敵を倒し、ステータスポイントを獲得



ステータスポイントを消費し、ステータスを上げる



ステータスをあげ、ボスを倒す



# 技術紹介①：ステータス

獲得したステータスポイントステータスを上げることができます。半分まで上げると、カスタムボーナスを取得でき、ステータスを上げる良さとして実装しました。上げた分のステータスを、csvで保存しています。元々のステータスに上げた分のステータスを足しています。

↓ステータス関係の情報を出力・読み込みを制作しました。

```
//強化ステータスを読み込む
void LoadCsv::LoadStatusUp(PlayerStatus::StatusUpValue& data)
{
    std::ifstream ifs(kStatusUpFileName);
    std::string line;
    std::vector<std::string> strvec;
    m_data.clear();

    while (std::getline(ifs, line))
    {
        strvec = split(line, ',');
        const char* str = strvec[0].c_str();

        // アニメーション情報を設定
        std::string animName = strvec[0];
        try
        {
            data.StatusPoint = std::stoi(strvec[0]);
            data.hpUp = std::stoi(strvec[1]);
            data.hpUpTimes = std::stoi(strvec[2]);
            data.attackPowerUp = std::stoi(strvec[3]);
            data.attackPowerUpTimes = std::stoi(strvec[4]);
            data.magicAttackPowerUp = std::stoi(strvec[5]);
            data.magicAttackPowerUpTimes = std::stoi(strvec[6]);
            data.defensePowerUp = std::stoi(strvec[7]);
            data.defensePowerUpTimes = std::stoi(strvec[8]);
            data.specialMoveGauge = std::stoi(strvec[9]);
            data.customBonus = std::stoi(strvec[10]);
        }

        catch (const std::invalid_argument& e)
        {
        }
    }
}
```

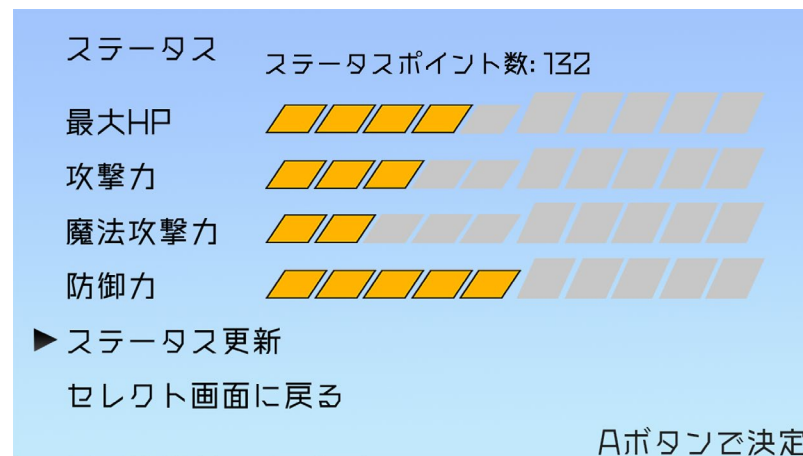
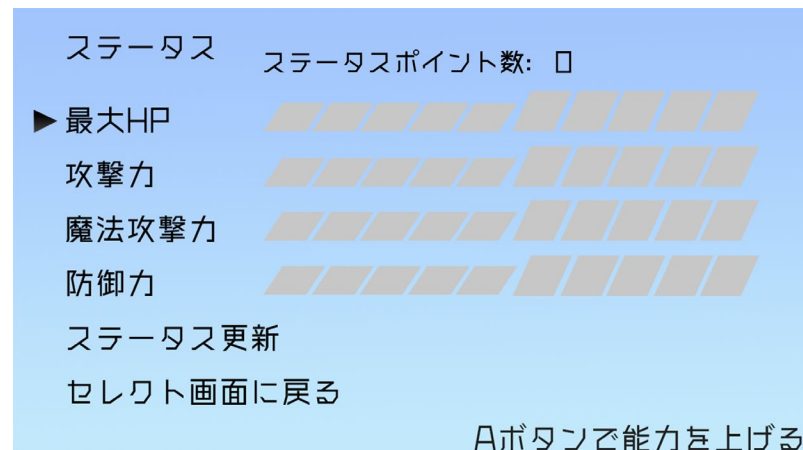
```
//データをCSVに出力
void PlayerStatus::StatOutput(int statusUpPoint)
{
    m_statusPoint = m_statusPoint + statusUpPoint;

    //出力したいファイル名を指定する
    std::ofstream outfile("data/csv/StatusUP.csv");

    std::vector<std::tuple<const char*, const char*, const char*, const char*, const char*, const char*,
        const char*, const char*, const char*, const char*, const char*>> nameData;
    std::vector<std::tuple<int, int, int, int, int, int, int, int, int, int, int>> data;

    //出力したいデータを作る
    nameData.push_back(std::make_tuple("ステータスポイント", "HPUP", "HPUP段階数", "攻撃力UP", "攻撃力UP段階数", "魔法攻撃力UP",
        "魔法攻撃力UP段階数", "防御力UP", "防御力UP段階数", "必殺技ゲージ", "カスタムボーナスの説明回数"));
    data.push_back(std::make_tuple(m_statusPoint, m_hpUp, m_attackUpTimes, m_attackUp, m_attackUpTimes, m_magicAttackUp,
        m_magicAttackUpTimes, m_defensePowerUp, m_defensePowerUpTimes, m_specialMoveGauge, m_customBonus));

    //データの出力
    for (auto& a : nameData) {
        for (auto& b : data) {
            //ファイルの出力
            outfile << a << "," << b << "\n";
        }
    }
    outfile.close();
}
```



# 技術紹介②：モブ敵の追加のしやすさ

モブ敵は、characterBaseを親クラスとしてつくりあげています。  
共通の処理の部分は、characterBaseにあり、その敵だけの処理する  
コードを書くことで追加することができます。

↓ characterBase.hから 共通して行う処理

```
/// <summary>
/// シャドウマップへの描画の準備
/// </summary>
void DrawShadowModel();

///アニメーションの進行
///ループしたかどうかを返す
/// <summary>
/// アニメーションを変更
/// </summary>
/// <param name="animName"></param>
void ChangeAnim(std::string animName);

/// <summary>
/// アニメーションを更新
/// </summary>
void UpdateAnim();

/// <summary>
/// アニメーションの総再生時間を取得する
/// </summary>
float GetAnimTotalTime(std::string animName);
```

```
/// <summary>
/// 向いている方向の更新
/// </summary>
/// <param name="playerpos"></param>
void UpdateAngle(VECTOR playerpos);

/// <summary>
/// プレイヤーの攻撃が当たった時の処理
/// </summary>
/// <param name="attackStart"></param>
/// <param name="attackEnd"></param>
/// <param name="radius"></param>
/// <param name="attackPower"></param>
void HitPlayerAttack(VECTOR attackStart, VECTOR attackEnd, float radius, int attackPower);

/// <summary>
/// 死んだときの動き
/// </summary>
void Die();
```

# 技術紹介③：ステートパターン

ステートパターンを使い、プレイヤーの場合は押されたボタン、敵は、プレイヤーとの距離によって行動が変わるようになっています。敵は、種類ごとに動きが違うので種類が何かを受け取りその種類ごとに行動を変えています。

↓ PlayerStateWalk.cppから

```
//走っている状態から歩く状態に変更
//RBボタンを押すことで歩く状態に変更
if (pad.IsPress("RB") && (pad.IsPress("left") || pad.IsPress("right") || pad.IsPress("up") || pad.IsPress("down")))
{
    m_nextState = std::make_shared<PlayerStateRun>(m_pPlayer);
    auto state = std::dynamic_pointer_cast<PlayerStateRun>(m_nextState);
    state->Init();
    return;
}

//歩いている状態から待機状態に変更
else if (pad.IsRelease("left") || pad.IsRelease("right") || pad.IsRelease("up") || pad.IsRelease("down"))
{
    m_nextState = std::make_shared<PlayerStateIdle>(m_pPlayer);
    auto state = std::dynamic_pointer_cast<PlayerStateIdle>(m_nextState);
    state->Init();
    return;
}

//歩いている状態から攻撃状態に変更
if (pad.IsPress("A"))
{
    m_nextState = std::make_shared<PlayerStateAttack>(m_pPlayer);
    auto state = std::dynamic_pointer_cast<PlayerStateAttack>(m_nextState);
    state->Init();
    return;
}

//歩いている状態から魔法攻撃状態に変更
if (pad.IsTrigger("B"))
{
    m_nextState = std::make_shared<PlayerStateMagicAttack>(m_pPlayer);
    auto state = std::dynamic_pointer_cast<PlayerStateMagicAttack>(m_nextState);
    state->Init();
    return;
}
```



# 制作作品 ②

・ GitHub <https://github.com/Remy-1216/MonsterCrusher>

- ・ 作品名 MonsterCrusher
- ・ ジャンル 3Dアクションゲーム
- ・ 開発環境 C++,DxLib
- ・ 対応機種 Windows
- ・ 制作期間 4か月
- ・ 担当 モデル・モーション以外すべて
- ・ ゲーム内容

敵を倒し、ステージを進めていく

3Dアクションゲームです。  
残り敵数を倒すと、柵の先に  
進むことができます。





# 制作作品③

・ GitHub <https://github.com/Remy-1216/Mazokunomori>

- ・ 作品名 魔族の森
- ・ ジャンル 2D 避けゲー
- ・ 開発環境 C++,DxLib
- ・ 対応機種 Windows
- ・ 制作期間 4か月
- ・ 担当 イラスト・アニメーション  
以外すべて

## ・ ゲーム内容

30秒間の間色々なモンスター達から避けるゲームです。全4ステージあり、自分で遊びたいステージを選べます。

